

# PROJECT REPORT

on

## SIMPLE COMMAND-LINE CALCULATOR

Developed in Python

Submitted by

**Deepanshu Atre**

Registration No.: 26BAI10905

Branch: AI & ML

Year: 2026

## 1. Abstract

This project presents a simple command-line calculator developed in Python. The program accepts an arithmetic expression of the form "number operator number" from the user, evaluates it, and displays the result. It supports addition, subtraction, multiplication and division, and handles common errors such as division by zero, unknown operators and invalid numeric input without crashing. The program runs in a continuous loop until the user chooses to quit.

## 2. Introduction

A calculator is one of the most fundamental programs used to learn the basics of programming. It involves taking input, processing it with conditional logic, handling errors and producing output. This project implements these ideas in a compact and readable Python program that runs in the terminal and needs no external libraries.

## 3. Objectives

- To design a program that performs basic arithmetic operations on two numbers.
- To practise the use of functions, conditional statements and loops in Python.
- To implement proper exception handling so that invalid input does not terminate the program.
- To provide a simple, user-friendly command-line interface.

## 4. Tools and Technologies

| Component            | Details                                              |
|----------------------|------------------------------------------------------|
| Programming language | Python 3 (3.6 or higher, because f-strings are used) |
| Libraries            | None – only built-in Python features are used        |
| Interface            | Command line / terminal                              |
| Editor / IDE         | Any text editor or Python IDE (e.g. IDLE, VS Code)   |

## 5. System Design

### 5.1 Program Structure

The program is divided into two functions. This separation keeps the calculation logic independent from the user interface, which makes the code easier to read, test and extend.

| Function            | Description                                                                                                                         |
|---------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| calculate(a, op, b) | Takes two numbers and an operator, performs the operation and returns the result. Raises ZeroDivisionError for division by zero and |
|                     |                                                                                                                                     |

main()

ValueError for an unknown operator.

## 5.2 Algorithm

- Start the program and display the instructions.
- Read an expression from the user and remove extra whitespace.
- If the input is "q", print a goodbye message and stop.
- Split the input into parts. If there are not exactly three parts, show a format hint and ask again.
- Convert the first and third parts to floating-point numbers.
- Call calculate() with the two numbers and the operator.
- Print the result, or print an error message if an exception occurs.
- Return to reading the next expression.

## 6. Implementation

### 6.1 Supported Operations

| Operator | Operation      | Example           |
|----------|----------------|-------------------|
| +        | Addition       | 5 + 3 gives 8.0   |
| -        | Subtraction    | 9 - 12 gives -3.0 |
| *        | Multiplication | 7 * 6 gives 42.0  |
| /        | Division       | 10 / 4 gives 2.5  |

### 6.2 Error Handling

The program handles the following situations without stopping:

- Division by zero – a ZeroDivisionError is raised with a clear message.
- Unknown operator – a ValueError is raised naming the invalid operator.
- Non-numeric input – float() raises a ValueError, which is caught in main().
- Wrong format – if the input does not contain exactly three parts, the user is asked to re-enter it.

### 6.3 Source Code

```
def calculate(a, op, b):
    if op == '+':
        return a + b
    elif op == '-':
        return a - b
    elif op == '*':
        return a * b
    elif op == '/':
        if b == 0:
            raise ZeroDivisionError("Cannot divide by zero")
        return a / b
    else:
        raise ValueError(f"Unknown operator: {op}")
```

```

def main():
    print("Simple Calculator")
    print("Operators: + - * /")
    print("Type 'q' to quit\n")

    while True:
        expr = input("Enter expression (e.g. 5 + 3): ").strip()
        if expr.lower() == 'q':
            print("Goodbye!")
            break

        parts = expr.split()
        if len(parts) != 3:
            print("Please enter in the form: number operator number\n")
            continue

        num1_str, op, num2_str = parts
        try:
            num1 = float(num1_str)
            num2 = float(num2_str)
            result = calculate(num1, op, num2)
            print(f"Result: {result}\n")
        except ValueError as e:
            print(f"Error: {e}\n")
        except ZeroDivisionError as e:
            print(f"Error: {e}\n")

if __name__ == "__main__":
    main()

```

## 7. Testing and Results

The program was run with a range of valid and invalid inputs. The observed outputs are shown below.

| Input   | Expected behaviour   | Actual output                                    |
|---------|----------------------|--------------------------------------------------|
| 5 + 3   | Addition             | Result: 8.0                                      |
| 10 / 4  | Division             | Result: 2.5                                      |
| 7 * 6   | Multiplication       | Result: 42.0                                     |
| 9 - 12  | Negative result      | Result: -3.0                                     |
| 8 / 0   | Division error       | Error: Cannot divide by zero                     |
| 2 ^ 3   | Unsupported operator | Error: Unknown operator: ^                       |
| abc + 1 | Invalid number       | Error: could not convert string to float: 'abc'  |
| 5 +     | Wrong format         | Please enter in the form: number operator number |
| q       | Quit                 | Goodbye!                                         |

All test cases produced the expected results, and the program continued to run

normally after every error.

## 8. Limitations

- Only one operation can be evaluated at a time; expressions such as  $2 + 3 * 4$  are not supported.
- The number, operator and second number must be separated by spaces.
- Brackets, powers, modulus and other advanced functions are not available.
- Results are always displayed as floating-point values (for example 8.0 instead of 8).

## 9. Future Scope

- Parse full expressions with operator precedence and brackets.
- Add more operations such as power, modulus, square root and percentage.
- Store a history of calculations.
- Develop a graphical interface using Tkinter or a web-based version.
- Write unit tests for the `calculate()` function using `unittest` or `pytest`.

## 10. Conclusion

The Simple Command-Line Calculator meets its objectives. It performs the four basic arithmetic operations, validates user input and recovers from errors gracefully. The project helped in understanding functions, control flow, string handling and exception handling in Python, and it provides a solid base on which more advanced calculator features can be built.

## 11. References

- Python Software Foundation, Python 3 Documentation – <https://docs.python.org/3/>
- Python Tutorial: Errors and Exceptions – <https://docs.python.org/3/tutorial/errors.html>